

COMPUTER GRAPHICS LAB

Common Programming Assignments | Total 50 Marks

Submission & Implementation

- Assignment 1: 25 marks; Assignment 2: 25 marks. No presentation or viva is required for these two assignments.
- Submit a complete, runnable project. Evaluation will be based on running the application and inspecting the source code.
- HTML5 Canvas + JavaScript or Python with a suitable graphics library is recommended; another appropriate environment may also be used.
- HTML: submit one self-contained .html file, or a simple folder with HTML/CSS/JS files. Python: include source files and identify any required external library.
- Transformations must be applied to the drawn geometry/coordinates. CSS-only or image-level transforms do not satisfy the graphics-transformation requirement.

Assignment 1

Interactive Analog Clock

25 marks

Objective

Use 2D primitives and coordinate transformations to build a live analog clock with animation, rotation, translation, and scaling.

REQUIRED FUNCTIONALITY

- Draw the clock from graphics primitives (for example circles/arcs, lines, points, polygons and text); do not use a pre-rendered clock image.
- Show the computer's current local time. Hour, minute and second hands must update automatically and rotate to the correct angles.
- Allow the clock to be translated/repositioned within the drawing area, preferably by mouse/touch dragging or equivalent controls.
- Allow the clock to be scaled/zoomed while preserving proportions; mouse-wheel zoom or an equivalent control is acceptable.
- The clock should remain readable, stable and visually coherent during animation and interaction.

ENHANCEMENTS

Useful additions may include manual time adjustment, themes, 12/24-hour or digital companion display, tick labels, smooth second-hand motion, reset/center controls, fullscreen mode, or other creative interactions. These are examples, not a fixed checklist.

MARKING SCHEME

Primitive-based clock 4 marks Clock face and essential elements are constructed from appropriate graphics primitives and coordinates.	Real-time clock & rotation 5 marks Current time is correct and the hands rotate/animate accurately from time values.
Translation 4 marks The clock can be moved by transforming the drawing position/coordinates.	Scaling / zoom 4 marks The clock can be resized through a proper scale transformation while keeping proportions.
Graphics implementation 3 marks Drawing and transformations are logically implemented, stable and readable in the source code.	Presentation & enhancements 5 marks Visual quality, usability, smooth interaction, and useful or creative additional features.

CORE IMPLEMENTATION: 20

PRESENTATION & ENHANCEMENTS: 5

TOTAL: 25

Assignment 2

Interactive 3D Cube and Transformations

25 marks

Objective

Demonstrate a basic 3D graphics pipeline: define 3D geometry, apply transformations, project it into 2D screen space, and render an interactive cube.

INTERFACE LAYOUT

- Use a large drawing viewport on the left side of the application window/page.
- Use an organized control panel on the right side for transformation and display controls.

CORE FUNCTIONALITY

- Define and render a cube from 3D data using vertices/edges, polygonal faces, triangles, or another clear geometric representation.
- Provide independent X-, Y- and Z-axis rotation controls, preferably sliders. Rotation must transform the 3D coordinates.
- Provide a scaling control that changes the cube size through a 3D scale transformation.
- Apply perspective projection so the 3D cube is projected into the 2D viewport; the code should clearly reflect the path from 3D coordinates to screen coordinates.
- Provide a simple camera/viewpoint control so the cube can be viewed from different vertical or angular positions.
- Make the faces easy to distinguish using face colors, simple depth-based shading, basic directional shading, or a similar technique.

SUGGESTED ADDITIONAL FEATURES

- Auto-rotate toggle so the cube rotates continuously when enabled.
- Wireframe / shaded toggle to switch between edge-only and filled/shaded views.
- Other useful additions: reset view, pause, rotation-speed control, field-of-view or camera-distance control, projection parameters, fullscreen mode, responsive layout, and similar creative improvements.

Implementation note

- High-level graphics libraries are not prohibited, but the core marks require an understandable implementation of cube geometry, transformations, projection and screen mapping. A library call that completely hides these steps may not demonstrate the intended learning outcome.

MARKING SCHEME

Cube geometry & rendering 4 marks A correct cube is defined from 3D geometry and rendered reliably in the viewport.	X / Y / Z rotation 6 marks Independent rotation around all three axes is correctly implemented using transformed 3D coordinates.
Scaling 2 marks Interactive 3D scaling is implemented correctly.	Perspective projection 4 marks 3D points are projected into 2D screen coordinates with a working perspective view.
Camera & surface cues 4 marks Viewpoint can be adjusted and faces are visually distinguishable through simple shading/color/depth cues.	Presentation & extras 5 marks Control-panel quality, auto-rotate, wireframe/shaded mode, smooth interaction, and other useful enhancements.

CORE IMPLEMENTATION: 20

PRESENTATION & ENHANCEMENTS: 5

TOTAL: 25

A polished interface is encouraged, but correct graphics concepts and a working implementation remain the priority.